Loan Service

Code Email for Engineers

Note to the candidates

Thanks for your interest in joining Amartha.
We expect all engineers to be able to write and have nuanced conversation around code. We ask you to solve a small simple problem around the problem statements we have in Amartha, so as to understand how you solve problems and implement solutions in code.
There are 3 problems in the following email. Please pick any one that interests you and think best highlights your skills.
Please post the code that solves the specific problem. We are happy to receive the code as a ZIP file of the folder(git please) or an online link to the code repository in the email. And we will review it in a week and respond back.
While implementing please think about simplicity of the code, and clarity for the reviewer(an Amartha engineer). Please make any reasonable assumption that is not explicit in the problem statement.
We are looking for quality of code, and the completeness of the solution(for the problem that is specified). We'd love to look at the tests esp for the basic requirements on the problems.

Proposed Code Problems

Example 1: Billing Engine (System Design and abstraction)

We are building a billing system for our Loan Engine. Basically the job of a billing engine is to provide the

- Loan schedule for a given loan(when am i supposed to pay how much)
- Outstanding Amount for a given loan
- Status of weather the customer is Delinquent or not

We offer loans to our customers a 50 week loan for Rp 5,000,000/- , and a flat interest rate of 10% per annum.
This means that when we create a new loan for the customer(say loan id 100) then it needs to provide the billing schedule for the loan as
W1 : 110000
W2: 110000
W3: 110000
...
W50: 110000

_Amartha__Code_test_Engineering.pdf

Loan Service (system design and abstraction)

📍 we are building a loan engine. A loan can a multiple state: proposed , approved, invested, disbursed. the rule of state:

1. proposed is the initial state (when loan created it will has proposed state):
2. approved is once it approved by our staff.
1. a approval must contains several information:
1. the picture proof of the a field validator has visited the borrower
2. the employee id of field validator
3. date of approval
2. once approved it can not go back to proposed state
3. once approved loan is ready to be offered to investors/lender
3. invested is once total amount of invested is equal the loan principal
1. loan can have multiple investors, each with each their own amount
2. total of invested amount can not be bigger than the loan principal amount
3. once invested all investors will receive an email containing link to agreement letter (pdf)
2. disbursed is when is loan is given to borrower.
1. a disbursement must contains several information:
1. the loan agreement letter signed by borrower (pdf/jpeg)
2. the employee id of the field officer that hands the money and/or collect the agreement letter
3. date of disbursement

movement between state can only move forward, and a loan only need following information:

● borrower id number
● principal amount
● rate, will define total interest that borrower will pay
● ROI return of investment, will define total profit received by investors
● link to the generated agreement letter

design a RESTFful api that satisfy above requirement.

我们正在构建一个贷款引擎。一笔贷款可以有多种状态：提议的、批准的、

投入、支出。国家统治：

1.提议是初始状态（当贷款创建时，它将具有提议状态）：

2. 批准是指经我们工作人员批准后。

1. 批准必须包含以下几项信息：

1. 现场验证员拜访过借款人的图片证明

2. 字段验证器的员工 ID

3. 批准日期

2.一旦批准就不能回到提议状态

3. 一旦获得批准，贷款就可以提供给投资者/贷方

3. 投资总额等于贷款本金后即为投资

1.贷款可以有多个投资者，每个投资者都有自己的金额

2、投资总额不能大于贷款本金

3. 投资后，所有投资者都会收到一封包含链接的电子邮件

协议书（pdf）

2. 支付是指向借款人提供贷款的时间。

1. 付款必须包含以下几项信息：

1. 借款人签署的贷款协议书（pdf/jpeg）

2. 交钱的现场官员的员工 ID 和/或

收集协议书

3. 付款日期

国家之间的流动只能向前，贷款只需要跟随

信息：

● 借款人身份证号码

●本金金额

● 利率，将定义借款人将支付的总利息

● 投资回报率（ROI），将定义投资者获得的总利润

● 生成的协议书的链接

设计一个满足上述要求的RESTFful api。

Functional Requirements

1. Loan Creation

系统需要支持创建贷款，并管理贷款的完整生命周期。

创建贷款时需要提供：

Field	Description	Required
borrower_id	借款人 ID	Yes
principal_amount	贷款本金	Yes
interest_rate	借款利率	Yes
roi	投资回报率	Yes
agreement_url	生成的贷款协议 PDF 链接	Yes

创建成功后，贷款初始状态必须为 `PROPOSED`。

2. Loan State Management

贷款包含四种状态：

PROPOSED(Created)

APPROVED(Approved by Staff)

INVESTED(Fully Funded)

DISBURSED disbursed已拨款(Funds Released)

Business Rules

- 贷款只能按照上述顺序转换状态。
- 不允许跳过状态。
- 不允许状态回退。
- 状态转换必须满足对应的业务条件。
- `DISBURSED` 是本题的终态。

3. Loan Approval

只有 `PROPOSED` 状态的贷款允许审批。

审批必须包含：

- Field validator employee ID（现场审核人员 ID）
- Visit proof（现场访问借款人的照片证明）

- Approval date（审批日期）

审批成功后：

`PROPOSED → APPROVED`

4. Loan Investment

贷款审批后，可以向多个投资者开放投资。

Business Rules

- 只有 `APPROVED` 状态允许新增投资。
- 一笔贷款可以有多个投资者。
- 每个投资者可以投入不同金额。
- 总投资金额不能超过贷款本金。
- 当累计投资金额等于贷款本金时，自动变更为 `INVESTED`。
- 满额投资后，需要向所有投资者发送包含贷款协议 PDF 链接的邮件。

核心约束：

$$0 \leq \sum_{i=1}^{n} Investment_i \leq Principal$$

当：

$$\sum_{i=1}^{n} Investment_i = Principal$$

贷款状态转换为 `INVESTED`。

5. Loan Disbursement

只有 `INVESTED` 状态允许放款。

放款需要：

- Borrower-signed loan agreement（借款人签署的协议）
- Field officer employee ID（放款经办人员 ID）
- Disbursement date（放款日期）

放款成功后：

`INVESTED → DISBURSED`

6. Additional Functional Requirements

以下属于我们补充的工程功能，并非题目明确要求：

Requirement	Description
Loan Query	查询贷款详情和当前状态
Investment Query	查询投资记录
Idempotent Investment	防止重复投资请求
Status History	记录贷款状态变化
Notification Tracking	跟踪邮件通知状态

Non-functional Requirements

1. Consistency

Critical

金融业务最重要的是数据正确性。

Requirements

- 投资总额不能超过本金。
- 投资记录与贷款累计投资金额必须一致。
- 贷款状态与业务数据必须一致。
- 审批、投资、放款的相关数据库操作必须具有原子性。

Solution

- PostgreSQL ACID Transactions
- Row-level Locking
- Database Constraints
- Atomic Updates

2. Concurrency Safety

多个投资者可能同时投资同一笔贷款。

系统必须保证：

- 不发生 Over-investment（超额投资）。
- 不发生 Lost Update（更新丢失）。

- 不出现重复的满额状态转换。

Solution

使用 PostgreSQL `SELECT FOR UPDATE` 锁定贷款记录，并在事务内完成投资金额校验和更新。

3. Reliability

Requirements

- 事务失败时必须回滚。
- 不允许出现部分成功的投资操作。
- 邮件发送失败不应破坏已提交的投资数据。
- 重复请求不能重复产生业务效果。

Solution

- Database Transaction
- Idempotency Key
- Transactional Outbox（推荐增强方案）

4. Performance

题目没有提供明确的 QPS 或延迟要求。

因此不预先承诺生产级 SLA。

Design Goals

- 使用主键和外键索引优化查询。
- 使用数据库连接池。
- 设置合理的请求超时。
- 避免长时间持有数据库锁。
- 不在数据库事务中调用外部邮件服务。

可以在实现后通过基准测试验证性能。

5. Scalability

采用 Stateless REST API（无状态接口）。

多个应用实例可以共享 PostgreSQL，但所有并发正确性由数据库事务保证，而不是依赖进程内锁。

本次作业不实现自动扩容或分布式部署。

6. Security

Requirements

- Validate all inputs.
- Prevent SQL injection.
- Protect sensitive borrower information.
- Restrict approval and disbursement operations to authorized staff.
- Restrict investments to authenticated investors.

Solution

- Parameterized SQL
- Request Validation
- Sensitive Data Masking
- Authentication / Authorization Interface

完整的身份认证和权限系统不属于原题范围，生产环境应集成可信的身份服务。

7. Observability

使用 Go `slog` 输出结构化日志。

建议记录：

- Request ID
- Loan ID
- Operation
- Duration
- Error Code
- Transaction Result

不在日志中输出借款人身份证号、协议内容等敏感数据。

8. Maintainability

- Clear separation of concerns
- Small and testable functions
- Explicit business rules
- Dependency injection
- Unit tests and integration tests
- Consistent error handling

Core Entities

loans

贷款主表。

Column	Type	Constraint
id	BIGINT	PK
borrower_id	VARCHAR(64)	NOT NULL
principal_amount	BIGINT	> 0
invested_amount	BIGINT	DEFAULT 0
interest_rate_bps	INTEGER	>= 0
roi_bps	INTEGER	>= 0
agreement_url	TEXT	NOT NULL
status	VARCHAR(20)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL
updated_at	TIMESTAMPTZ	NOT NULL

Constraints

- `invested_amount >= 0`
- `invested_amount <= principal_amount`
- `status IN ('PROPOSED', 'APPROVED', 'INVESTED', 'DISBURSED')`
- `INVESTED` 和 `DISBURSED` 必须已经满额。

`invested_amount` 是投资明细的汇总缓存，必须在同一个事务中维护。

loan_approvals

Column	Type	Constraint
id	BIGINT	PK
loan_id	BIGINT	FK, UNIQUE
validator_employee_id	VARCHAR(64)	NOT NULL
visit_proof_url	TEXT	NOT NULL
approved_at	TIMESTAMPTZ	NOT NULL

loan_investments

Column	Type	Constraint
id	BIGINT	PK
loan_id	BIGINT	FK
investor_id	VARCHAR(64)	NOT NULL
amount	BIGINT	> 0
idempotency_key	VARCHAR(100)	UNIQUE
created_at	TIMESTAMPTZ	NOT NULL

Indexes:

- `idx_investments_loan_id`
- Unique index on `idempotency_key`

loan_disbursements

Column	Type	Constraint
id	BIGINT	PK
loan_id	BIGINT	FK, UNIQUE
officer_employee_id	VARCHAR(64)	NOT NULL
signed_agreement_url	TEXT	NOT NULL
disbursed_at	TIMESTAMPTZ	NOT NULL

loan_status_history

Column	Type	Constraint
id	BIGINT	PK
loan_id	BIGINT	FK
from_status	VARCHAR(20)	Nullable
to_status	VARCHAR(20)	NOT NULL
action	VARCHAR(30)	NOT NULL
created_at	TIMESTAMPTZ	NOT NULL

用于审计状态变更，不作为当前状态的来源。

notification_outbox（Optional）

Column	Type	Description
id	BIGINT	PK
event_type	VARCHAR(50)	事件类型
aggregate_id	BIGINT	Loan ID
payload	JSONB	事件数据
status	VARCHAR(20)	PENDING / SENT / FAILED
attempts	INTEGER	重试次数
next_attempt_at	TIMESTAMPTZ	下次尝试时间
created_at	TIMESTAMPTZ	创建时间

API

采用 RESTful API，版本前缀：`/api/v1`

Method	Endpoint	Description
POST	/loans	Create a loan
GET	/loans/{id}	Get loan details
POST	/loans/{id}/approval	Approve a loan
POST	/loans/{id}/investments	Create an investment
GET	/loans/{id}/investments	List investments
POST	/loans/{id}/disbursement	Disburse a loan

Create Loan

`POST /loans`

Request:

代码块

```
{
  "borrower_id": "BORROWER001",
  "principal_amount": 5000000,
  "interest_rate_bps": 1000,
  "roi_bps": 800,
  "agreement_url": "https://example.com/loan.pdf"
```

```
}
```

Response `201 Created`:

```
代码块
{
  "id": 100,
  "status": "PROPOSED",
  "principal_amount": 5000000,
  "invested_amount": 0
}
```

Get Loan

`GET /loans/100`

Response:

```
代码块
{
  "id": 100,
  "borrower_id": "BORROWER001",
  "principal_amount": 5000000,
  "invested_amount": 2000000,
  "remaining_amount": 3000000,
  "status": "APPROVED"
}
```

Approve Loan

`POST /loans/100/approval`

Request:

```
代码块
{
  "validator_employee_id": "EMP001",
  "visit_proof_url": "https://example.com/proof.jpg",
  "approved_at": "2026-10-16T10:00:00Z"
}
```

Response:

```
代码块
1  {
2    "loan_id": 100,
3    "status": "APPROVED"
4  }
```

Create Investment

`POST /loans/100/investments`

Header:

`Idempotency-Key: investment-001`

Request:

```
代码块
1  {
2    "investor_id": "INVESTOR001",
3    "amount": 2000000
4  }
```

Response `201 Created`:

```
代码块
1  {
2    "investment_id": 1001,
3    "loan_id": 100,
4    "total_invested": 2000000,
5    "remaining_amount": 3000000,
6    "status": "APPROVED"
7  }
```

当累计投资额达到本金时，`status` 自动变为 `INVESTED`。

Disburse Loan

`POST /loans/100/disbursement`

Request:

```
代码块
1  {
```

```
  "officer_employee_id": "EMP002",
  "signed_agreement_url": "https://example.com/signed.pdf",
  "disbursed_at": "2026-10-16T12:00:00Z"
}
```

Response:

代码块

```
{
  "loan_id": 100,
  "status": "DISBURSED"
}
```

Error Handling

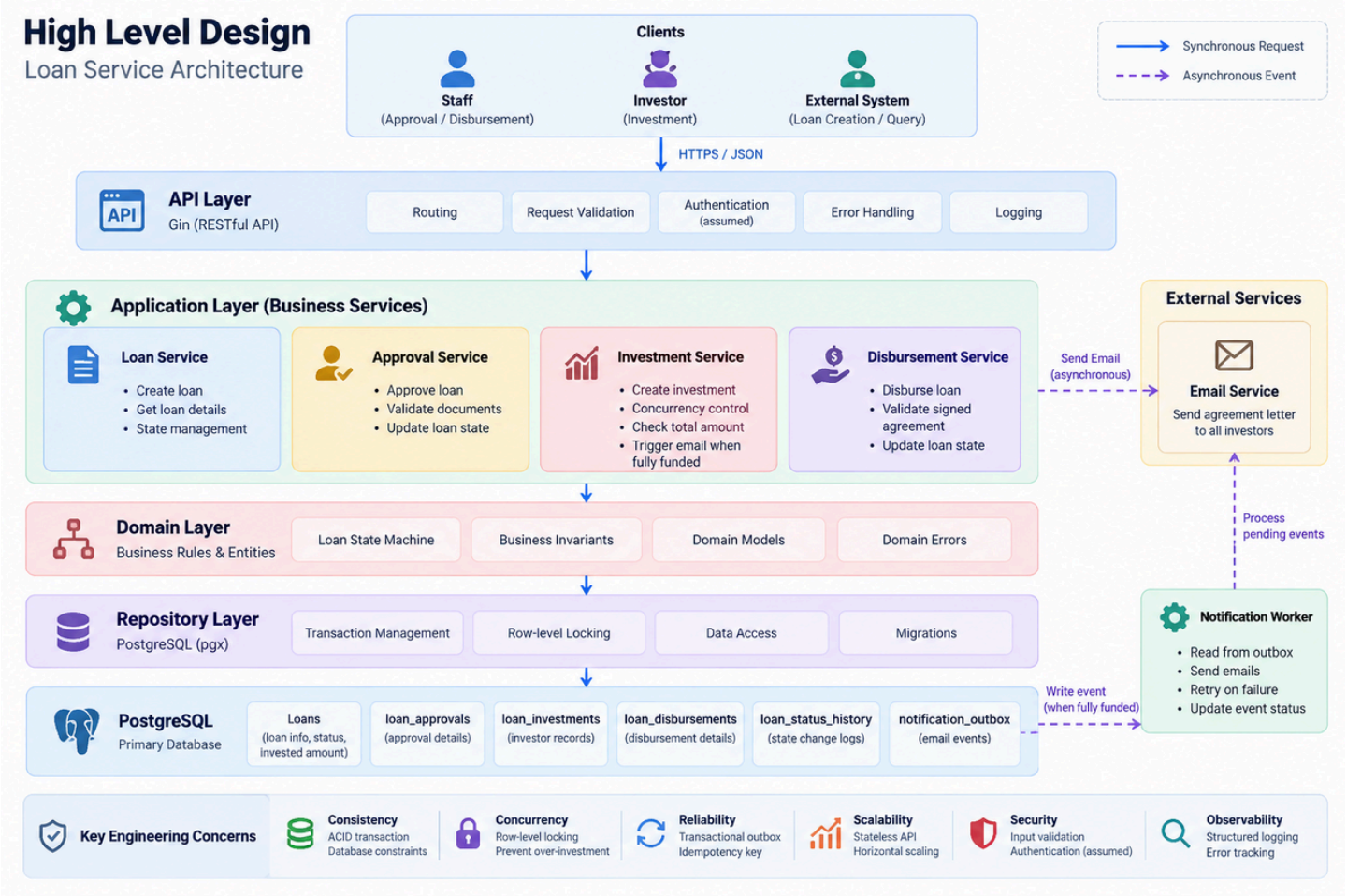
HTTP Code	Scenario
400	Invalid request
404	Loan not found
409	Invalid state transition, over-investment or idempotency conflict
422	Business validation failed
500	Internal server error

统一错误格式：

代码块

```
{
  "error": {
    "code": "INVALID_LOAN_STATE",
    "message": "Loan must be APPROVED before investment"
  }
}
```

High Level Design



下面按照你指定的 6 个章节，整理成一份适合 Amartha Senior Backend Engineer Take-home Assignment 的技术设计文档。

我会保留英文技术术语，使用中文解释设计决策。内容以 Amartha 第三题 Loan Service 为基础，明确区分题目要求和我们主动增加的工程设计。

Design Proposal · Loan Service · Go + PostgreSQL

1. Architecture Overview

采用 Modular Monolith（模块化单体）。

Client / API Consumer

HTTP Requests

HTTP Layer · Gin

Routing · Validation · Error Mapping

Application Layer

Loan Service

Approval Service

Investment Service

Disbursement Service

Domain Layer

State Machine · Business Invariants · Domain Errors

Repository Layer

pgx · SQL · Transaction Management

PostgreSQL

Notification Worker（Optional）

Outbox → Email Adapter

2. Technology Stack

Component	Technology
Language	Go
HTTP	Gin
Database	PostgreSQL
DB Driver	pgx/v5
Migration	golang-migrate
Logging	slog
Unit Testing	testing + testify
Integration Testing	testcontainers-go
API Documentation	OpenAPI 3.0
Containerization	Docker Compose

3. Project Structure

代码块

```
amartha-loan-service/
├── cmd/
│   └── server/
│       └── main.go                 # Application entry point
│
├── internal/
│   ├── domain/                     # Domain models & business rules
│   │   ├── loan.go                 # Loan entity & state machine
│   │   ├── investment.go           # Investment entity
```

```
10  │   │   └── errors.go                # Domain errors
11  │   │
12  │   ├── service/                     # Business logic
13  │   │   ├── loan_service.go          # Create, Get, Approve, Disburse
14  │   │   ├── investment_service.go    # Invest, concurrency, funding
15  │   │   └── notification_service.go  # Notify investors
16  │   │
17  │   ├── repository/                  # Data access abstraction
18  │   │   ├── repository.go            # Repository interfaces
19  │   │   └── postgres/
20  │   │       ├── loan.go
21  │   │       ├── investment.go
22  │   │       └── transaction.go
23  │   │
24  │   ├── handler/                     # REST API
25  │   │   ├── loan_handler.go
26  │   │   └── investment_handler.go
27  │   │
28  │   └── infrastructure/              # External dependencies
29  │       ├── database.go
30  │       └── email.go
31  │
32  ├── migrations/
33  │   ├── 001_init.up.sql
34  │   └── 001_init.down.sql
35  │
36  ├── tests/
37  │   └── integration/
38  │       └── investment_test.go
39  │
40  ├── docs/
41  │   ├── design.md
42  │   └── openapi.yaml
43  │
44  ├── docker-compose.yml
45  ├── Makefile
46  ├── go.mod
47  └── README.md
```

Deep Dives

1. Loan State Machine

Problem

贷款状态必须按照固定顺序转换，不允许非法跳转或回退。

Solution

将状态转换规则集中在 Domain 层。

代码块

```
type LoanStatus string

const (
    Proposed  LoanStatus = "PROPOSED"
    Approved  LoanStatus = "APPROVED"
    Invested  LoanStatus = "INVESTED"
    Disbursed LoanStatus = "DISBURSED"
)

func CanTransition(from, to LoanStatus) bool {
    switch from {
    case Proposed:
        return to == Approved
    case Approved:
        return to == Invested
    case Invested:
        return to == Disbursed
    default:
        return false
    }
}
```

状态检查只是第一层校验。

每个转换还需要对应的业务条件，例如投资满额、审批材料齐全等。

Design Decision

不使用复杂的状态机框架，因为只有四个状态，显式转换规则更容易理解和测试。

2. Concurrent Investment

Problem

多个投资者同时投资同一笔贷款，可能发生超额投资。

例如：

本金 Rp 5,000,000，当前已投资 Rp 4,000,000。

两个投资者同时投资 Rp 800,000。

如果没有并发控制，最终可能达到 Rp 5,600,000。

Solution

使用 PostgreSQL Transaction + Row-level Lock。

代码块

```
BEGIN;

SELECT id, principal_amount, invested_amount, status
FROM loans
WHERE id = $1
FOR UPDATE;

-- Validate status and remaining amount
-- Insert investment record
-- Update invested amount
-- Update status if fully funded

COMMIT;
```

Why Row-level Lock?

- 对同一笔贷款的投资进行串行化。
- 不同贷款可以并发处理。
- 不需要额外的 Redis 分布式锁。
- 事务回滚时数据库自动释放锁。

所有投资写入都必须遵守同一套锁与事务规则。

3. Transaction Boundary

投资操作的事务边界：

BEGIN TRANSACTION

Lock Loan Row

Validate Loan Status

Check Idempotency & Amount

Insert Investment

Update Invested Amount

Update Status if Fully Funded

Insert Status History / Outbox Event

COMMIT

只要其中任何数据库操作失败，整个事务回滚。

Invariant（不变量）：

$$ \text{Loan.InvestedAmount} = \sum \text{Committed Investments} $$

这个不变量通过事务内的统一写入路径维护，而不是依靠普通 CHECK 约束自动计算跨表合计。

4. Idempotency

Problem

客户端因为网络超时重试投资请求，可能产生重复投资。

Solution

客户端提供唯一的 `Idempotency-Key`。

服务端将其与请求内容关联并存储。

场景	行为
新 Key	正常创建投资
相同 Key、相同请求	返回已有结果
相同 Key、不同请求	409 Conflict
并发使用相同 Key	只允许产生一次业务效果

使用数据库唯一约束处理并发竞争，不能只依赖先查询再插入。

5. Reliable Email Notification

Problem

投资满额后需要通知所有投资者。

如果贷款状态更新成功，但邮件发送失败，应该怎么办？

Solution

推荐 Transactional Outbox Pattern。

Investment Fully Funded

Database Transaction

Update Loan + Insert Outbox Event

Outbox Worker

Email Service

邮件通知应按投资者生成收件任务，并考虑同一投资者多次投资时是否只发送一封邮件。

建议按 `loan_id + investor_id + notification_type` 做通知去重。

Outbox Worker 失败后可以重试，但仍需处理重复投递。

如果时间不足，可以先实现 Notification 接口及 Mock，并在 README 说明生产环境增强方案。

6. Auditability

金融系统需要能够追踪关键状态变更。

通过 `loan_status_history` 记录：

- Previous State
- New State
- Action
- Timestamp

审批与放款的员工信息分别保存在对应业务表。

状态历史用于审计，不允许客户端直接修改。

7. Testing Strategy

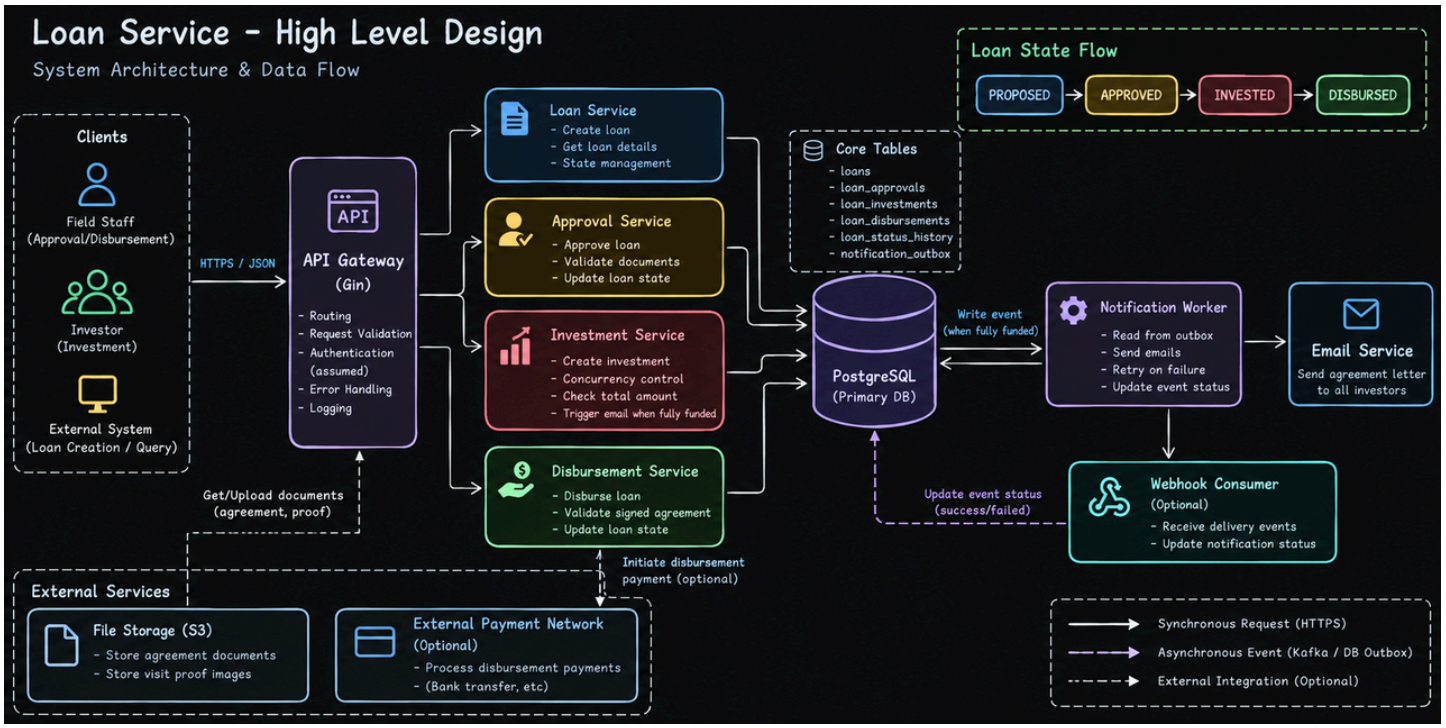
Test Category	Test Case	Expected Result
State Machine	PROPOSED → APPROVED	Success
State Machine	PROPOSED → DISBURSED	Rejected
Approval	Missing proof	Rejected
Investment	Amount exceeds principal	Rejected
Investment	Total equals principal	INVESTED
Concurrency	Two investments exceed remaining balance	At most one succeeds
Transaction	Database write failure	Full rollback
Idempotency	Same key retried	No duplicate investment
Disbursement	Loan not fully funded	Rejected
Notification	Fully funded	Notification generated
Notification	Email failure	Committed investment preserved

建议测试分为：

- Unit Tests：领域规则与状态机。
- Service Tests：业务编排和错误处理。
- Integration Tests：真实 PostgreSQL 事务、锁、并发和约束。

8. Trade-offs and Future Improvements

Final


Loan Service - High Level Design
System Architecture & Data Flow
Loan State Flow
PROPOSED
APPROVED
INVESTED
DISBURSED
Clients
Field Staff
(Approval/Disbursement)
Investor
(Investment)
External System
(Loan Creation / Query)
HTTPS / JSON
API
API Gateway
(Gin)
- Routing
- Request Validation
- Authentication
(assumed)
- Error Handling
- Logging
Loan Service
- Create loan
- Get loan details
- State management
Approval Service
- Approve loan
- Validate documents
- Update loan state
Investment Service
- Create investment
- Concurrency control
- Check total amount
- Trigger email when fully funded
Disbursement Service
- Disburse loan
- Validate signed agreement
- Update loan state
Core Tables
- loans
- loan_approvals
- loan_investments
- loan_disbursements
- loan_status_history
- notification_outbox
PostgreSQL
(Primary DB)
Write event
(when fully funded)
Notification Worker
- Read from outbox
- Send emails
- Retry on failure
- Update event status
Email Service
Send agreement letter
to all investors
Update event status
(success/failed)
Webhook Consumer
(Optional)
- Receive delivery events
- Update notification status
Get/Upload documents
(agreement, proof)
Initiate disbursement
payment (optional)
External Services
File Storage (S3)
- Store agreement documents
- Store visit proof images
External Payment Network
(Optional)
- Process disbursement payments
- (Bank transfer, etc)
Synchronous Request (HTTPS)
Asynchronous Event (Kafka / DB Outbox)
External Integration (Optional)